

colorlinks=true, unicode=true, linkcolor=black, citecolor=black, urlcolor=black, bookmarksopen=true



Západočeská univerzita v Plzni
Fakulta aplikovaných věd

KIV/PPR

Frameworky pro inferenci modelů neuronových sítí

Hana Hrkalová
hrkalovh@students.zcu.cz
A22N0076P

8. 12. 2024 Plzeň

1 Úvod

Tato práce představuje porovnání frameworků pro inferenci neuronových sítí na architekturách CPU a GPU. Cílem tohoto projektu bylo identifikovat a implementovat efektivní řešení pro nasazení předtrénovaných modelů se zaměřením na dosažení optimálního výkonu na obou platformách. To zahrnovalo vyhodnocení různých frameworků, včetně Frugally-Deep, TensorRT a dalších, implementaci samostatných optimalizovaných řešení pro inferenci na CPU a GPU a měření metrik inference. Následující části podrobně popisují proces výběru, podrobnosti implementace a analýzu výkonu vybraných řešení.

2 Analýza Frameworků

V této části je uveden přehled uvažovaných frameworků pro implementaci efektivní inference neuronových sítí.

2.1 FrugallyDeep

Frugally-Deep je pouze hlavičková knihovna určená pro rychlé a efektivní odvozování modelů hlubokého učení na procesorech. Zaměřuje se na minimalizaci závislostí a poskytuje jednoduché API pro načítání a spouštění modelů. [4]

- **Klíčové vlastnosti:** Optimalizováno pro výkon CPU, minimální závislosti.
- **Silné stránky:** Rychlá inference na CPU, snadná integrace.
- **Slabé stránky:** Bez podpory GPU, má méně pokročilých optimalizačních funkcí ve srovnání s jinými frameworky, nutnost JSON formátu modelu neuronových sítí, omezený pouze na Keras, nepodporuje Tensorflow a Pytorch
- **Význam pro projekt:** Frugally-Deep byl zvažován kvůli svému zaměření na výkon CPU a snadné použití. Nakonec byl vybrán jako primární framework pro inferenci na CPU.

2.2 Caffe

Caffe: Caffe (Convolutional Architecture for Fast Feature Embedding) je starší framework pro hluboké učení. Byl jeden z prvních frameworků, ve své době známý rychlostí a expresivní architekturou, zejména pro klasifikaci obrazu. [2]

- **Klíčové vlastnosti:** Expresivní architektura pro obrazové úlohy, C++ s rozhraním Python, podpora CPU a (především CUDA) GPU.
- **Silné stránky:** Historicky rychlý, zejména pro klasifikaci obrazu, modely a optimalizace jsou definovány konfigurací bez nutnosti tvrdého kódování.
- **Slabé stránky:** Vývoj již není aktivně udržován.

- **Význam pro projekt:** Caffé byl zvažován vzhledem k možnosti využití jak na CPU tak na GPU. Avšak vzhledem k jeho nedostatečné aktivní údržbě a dostupnosti modernějších a aktivněji vyvíjených frameworků s lepší podporou nebyl pro implementaci vybrán.

2.3 Tiny-cnn

Tiny-cnn: Tiny-cnn je knihovna pro hluboké učení bez závislostí napsaná v jazyce C++. Je určena pro vestavěné systémy a zařízení s omezenými zdroji. [33]

- **Klíčové vlastnosti:** Knihovna hlavičková, minimální závislosti, zaměřená na vestavěné systémy, zaměřená na CPU.
- **Silné stránky:** Malá velikost, snadná integrace, vhodná pro prostředí s omezenými zdroji.
- **Slabé stránky:** Omezené funkce ve srovnání s většími frameworky, primárně zaměřená na CPU s žádnou podporou GPU, vývoj není aktivně udržován.
- **Význam pro projekt:** Tiny-cnn byl zvažován kvůli své nenáročné povaze a zaměřením na inferenci CPU, což se shoduje s částí cílů projektu. Nedo- statek aktivní údržby jej však činily méně vhodným.

2.4 TensorRT

NVIDIA TensorRT: TensorRT je SDK vyvinuté společností NVIDIA pro vysoce výkonné inference hlubokého učení na grafických procesorech NVIDIA. Optimalizuje natrénované modely pro nasazení se zaměřením na maximalizaci propustnosti a minimalizaci latence. [32]

- **Klíčové funkce:** Optimalizováno pro grafické procesory NVIDIA (CUDA), podporuje různé formáty modelů (např. ONNX, TensorFlow, PyTorch), nabízí optimalizace, jako je kvantizace, slučování vrstev a automatické ladění jádra.
- **Silné stránky:** Výrazné zvýšení výkonu na grafických procesorech NVIDIA, zaměřuje zejména na rychlé a efektivní spuštění již natrénované sítě na GPU za účelem vygenerování výsledku (inference).
- **Slabé stránky:** Vazba na systém NVIDIA (vyžaduje grafické procesory NVIDIA), vyžaduje převod modelu na engine TensorRT.

- **Význam pro projekt:** TensorRT byl nakonec považován za jeden z hlavních kandidátů pro inferenci na GPU díky svému zaměření na výkon a optimalizaci pro hardware NVIDIA. [18]

2.5 TMVA Sofie

TMVA Sofie: TMVA (Toolkit for Multivariate Analysis) je projekt v rámci ROOT, který se používá především ve vysokoenergetické fyzice. Sofie (System for Optimised Fast Inference code Emit) je relativně nový přírůstek do TMVA, jehož cílem je poskytnout možnosti inference neuronových sítí. [28]

- **Klíčové vlastnosti:** Podporuje různé architektury neuronových sítí a má podporovat inferenci na CPU i GPU.
- **Silné stránky:** Inference na CPU i GPU, rychlost.
- **Slabé stránky:** Relativně nové a méně vyzrálé ve srovnání s jinými frameworky, Sofie se spouští pod systémem ROOT, omezená dokumentace a podpora komunity, podpora GPU (SYCL) zatím není plně integrována do oficiálních verzí, práce s nimi může být náročná kvůli omezeným zdrojům.
- **Význam pro projekt:** TMVA Sofie byla zkoumána jako potenciální řešení pro inferenci na CPU i GPU díky svému potenciálu podpory napříč platformami.

2.6 ONNX Runtime

ONNX Runtime je multiplatformní inferenční engine určený k spouštění modelů ve formátu ONNX (Open Neural Network Exchange). Zaměřuje se na umožnění interoperability mezi různými frameworky hlubokého učení a hardwarovými platformami. [19]

- **Klíčové funkce:** Podporuje spouštění modelů ONNX na různém hardwaru (CPU, GPU), multiplatformní (Windows, Linux, macOS), integruje se s různými frameworky (PyTorch, TensorFlow atd.), optimalizovaný pro přenositelnost.
- **Silné stránky:** Vynikající pro spouštění modelů natrénovaných v různých frameworkcích na různém hardwaru, zjednodušuje nasazení v různých prostředích, dobrá přenositelnost.

- **Slabé stránky:** Přidává další krok převodu (do ONNX).
- **Význam pro projekt:** ONNX Runtime byl jeden z hlavních kandidátů pro své multiplatformní schopnosti a schopnost spouštět modely z různých frameworků.

V této kapitole bylo představeno zhodnocení několika uvažovaných frameworků pro efektivní inferenci neuronových sítí na CPU a GPU. I když aplikace jako Caffé, Tiny-cnn nabízejí cenné funkce a možnosti, nebyly vybrány pro konečnou implementaci z důvodů, jako jsou nedostatek aktivní údržby, omezené sady funkcí a další. Nakonec byl pro inferenci na CPU vybrán Frugally-Deep díky své rychlosti a jednoduchosti, zatímco TensorRT byl vybrán pro inferenci na GPU díky svému vysoce optimalizovanému výkonu na hardwaru NVIDIA. Dále byla vybrána knihovna ONNX Runtime, která nabízí možnost spuštění inference na různých prostředích. TMVA Sofie byla zkoumána jak pro inferenci na CPU, tak na GPU.

3 Analýza counterů

Tato sekce hodnotí několik nástrojů pro analýzu hardwarových a softwarových výkonnostních čítačů, se zaměřením na jejich vhodnost pro profilování inference neuronových sítí na GPU a CPU.

3.1 Intel VTune Profiler:

- Klíčové vlastnosti: Komplexní nástroj pro analýzu výkonu procesorů Intel, podporuje různé techniky profilování (např. hotspots, analýza paměti, analýza souběžnosti), dokáže profilovat i některé aspekty GPU Intel. [5]
- Silné stránky: Výkonný nástroj pro analýzu výkonu CPU na architekturách Intel, široká škála funkcí profilování.
- Slabé stránky: Primárně zaměřený na hardware Intel, omezená podpora pro GPU NVIDIA, méně relevantní pro projekt zaměřený na výkon GPU NVIDIA.

3.2 AMD uProf (dříve CodeXL)

- Klíčové vlastnosti: Nástroj pro analýzu výkonu procesorů a grafických karet AMD, poskytuje přehled o využití CPU a GPU, přístupu k paměti a dalších metrikách výkonu. [1]
- Silné stránky: Komplexní profilování pro hardware AMD.
- Slabé stránky: Primárně zaměřený na hardware AMD, nerelevantní pro profilování GPU na NVIDIA.

3.3 NVIDIA Nsight Systems

- Klíčové vlastnosti: Nástroj pro analýzu výkonu celého systému pro CPU a GPU, poskytuje časovou osu aktivity aplikace, včetně spouštění jader CPU a GPU, přenosů paměti, volání API a dalších událostí. Podporuje různé techniky profilování.

- **Silné stránky:** Navržen speciálně pro hardware NVIDIA, poskytuje jednotný pohled na aktivitu CPU a GPU, uživatelsky přívětivé rozhraní, aktivně udržovaný společností NVIDIA, podporuje různé techniky profilování (včetně vzorkování a trasování), umožňuje korelaci událostí CPU a GPU.
- **Slabé stránky:** Primárně zaměřený na hardware NVIDIA. I když může profilovat aktivitu CPU i na procesorech jiných výrobců, hluboká integrace a optimalizace jsou pro systémy NVIDIA.

Nsight Systems je nejvhodnějším nástrojem pro tento projekt díky svému zaměření na hardware NVIDIA, sjednoceným možnostem profilování CPU a GPU a uživatelsky přívětivému rozhraní. Umožňuje komplexní pochopení interakce mezi CPU a GPU během inference neuronových sítí.

3.4 GpuPerfCounters

- **Klíčové vlastnosti:** Knihovna pro přímý přístup k výkonnostním čítačům GPU, poskytuje nízkoúrovňový přístup k hardwarovým datům o výkonu.
- **Silné stránky:** kontrola nad sběrem dat z výkonnostních čítačů.
- **Slabé stránky:** Vyžaduje více technických znalostí pro efektivní použití, zahrnuje psaní vlastního kódu pro sběr a analýzu dat, méně uživatelsky přívětivý než nástroje vyšší úrovně, jako je Nsight Systems.

Pro tento projekt, který se zaměřuje na výkon inference neuronových sítí na GPU NVIDIA a související aktivitu CPU, je NVIDIA Nsight Systems nejvhodnějším nástrojem. Jeho úzká integrace s hardwarem NVIDIA, kombinované profilování CPU/GPU, uživatelsky přívětivé rozhraní a aktivní údržba z něj činí nejlepší volbu pro dosažení komplexního pochopení chování systému. Zatímco jiné nástroje nabízejí cenné funkce, jsou buď zaměřeny na jiné výrobce hardware, nebo vyžadují vyšší úroveň technických znalostí. Nsight Systems poskytuje nejlepší poměr funkcionality, snadnosti použití a relevance k cílům projektu.

4 Implementace

Tato kapitola popisuje implementační detaily programu pro inferenci, který je napsán v jazyce C++. Zaměřuje se na klíčové komponenty, datové struktury a algoritmy použité k dosažení efektivního a spolehlivého inferenčního procesu.

4.1 Model

Pro účely semestrální práce byl využit vlastní model na klasifikaci ručně psaných čísel (MNIST). Tento model byl implementován v jazyce Python s využitím knihovny Keras, která poskytuje rozhraní pro tvorbu a trénování neuronových sítí.

Model musí být před jeho využíváním pro inferenci exportován. Pro inferenci na CPU a na GPU jsou vyžadovány různé formáty, a tak je zapotřebí i při exportu dbát na to, aby byl formát modelu konvertovatelný na formát ONNX (Open Neural Network Exchange) pro inferenci na GPU, nebo do formátu Frugally Deep JSON pro inferenci na CPU. Dále jsou popsány formáty modelu, které jsou konvertovatelné do požadovaných formátů pro inferenci.

- **CPU** - Pro inferenci na CPU je potřeba model uložit ve formátu Keras (.keras) a poté konvertovat pomocí Python skriptu, který se nachází v knihovně Frugally Deep. Konverze poté vypadá následujícím způsobem [3]:

```
python3 keras_export/convert_model.py \
    <path_to_input_.keras_model> \
    <path_to_output_.json_model>
```

Zdrojový kód 4.1: Export modelu pro Frugally Deep

- **GPU** - Pro inferenci na GPU je možné model exportovat do libovolného formátu kompatibilního s knihovnou Keras. Dále může být model konvertován pomocí Python knihovny tf2onnx [7].

4.2 Inference

Pro dosažení optimálního výkonu a efektivity inferenčního enginu byly zvoleny knihovny Frugally Deep, TensorRT a Onnx-Runtime. Tato kapitola analy-

zuje jejich silné stránky v kontextu požadavků aplikace, zejména poté porovnání a vyhodnocení výkonu jednotlivých knihoven.

4.2.1 Kompilace programu

Řešení je implementováno jako jeden program s přepínačem, který udává, zda má být program zkompilován a spuštěn na CPU s využitím knihovny Frugally Deep, TensorRT nebo Onnx-Runtime na CUDA.

```
set(FRUGALLY_DEEP "1")
set(TENSOR_RT "2")
set(ONNX_RUNTIME_CUDA "3")
if (${ACCELERATOR} MATCHES ${ONNX_RUNTIME_CUDA})
    set(ACCELERATE_ONNX_RUNTIME_CUDA 1)
elif (${ACCELERATOR} MATCHES ${TENSOR_RT})
    set(ACCELERATE_TENSOR_RT 1)
else ()
    set(ACCELERATE_FRUGALLY_DEEP 1)
endif ()
```

Zdrojový kód 4.2: Přepínač akcelerátoru

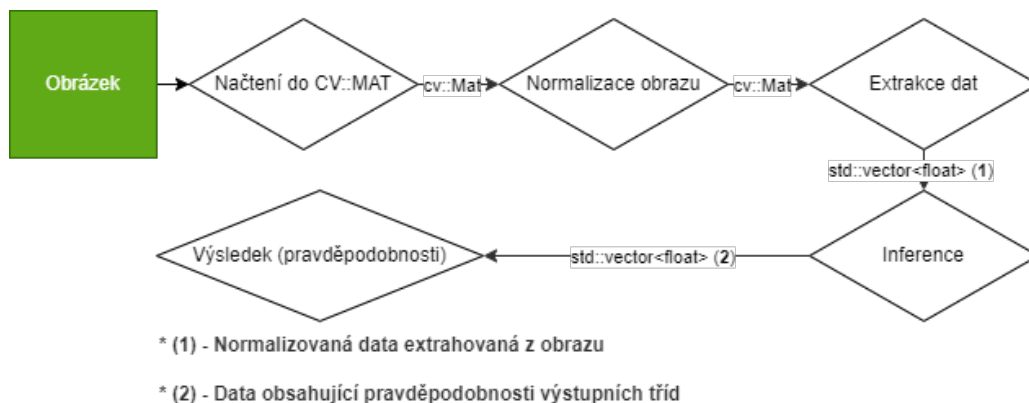
V kódu 4.2 lze vidět, že pro kompilaci programu pro CPU nebo GPU stačí pouze zahrnout při kompilaci argument `-DACCELERATOR` jako v kódu 4.3.

```
cmake --build "./onnx-runtime-build" -DACCELERATOR=3
```

Zdrojový kód 4.3: Kompilace programu pro spuštění pomocí ONNX-Runtime

4.2.2 Struktury

Pro inferenci obrázku je použito hned několik datových struktur, které se během běhu programu transformují, viz. obrázek 1.



Obrázek 1: Tok datových struktur

Z obrázku 1 lze vidět, že je prvně obrázek načten pomocí knihovny OpenCV do datové struktury `cv::Mat` jako jednorozměrný obrázek (černobílý). Poté je u obrázku změněna velikost na 28x28 a je normalizován (byty 0-255 jsou převedeny na typ `float` nabývající až 0-1). Dále jsou data vytažena do jednorozměrného pole a vložena jako vstupní argument pro inferenci. Jako výstup inference je jednorozměrné pole typu `float`, kde hodnoty nabývají od 0 do 1 a představují pravděpodobnost s jakou vstup nabývá výstupní třídy.

Struktury popsané výše jsou využity výhradně pro přenos dat a samotnou funkcionalitu programu. Dále se však v programu vyskytují struktury, které napomáhají k měření času, který samotná inference zabrala, viz. kód 4.4.

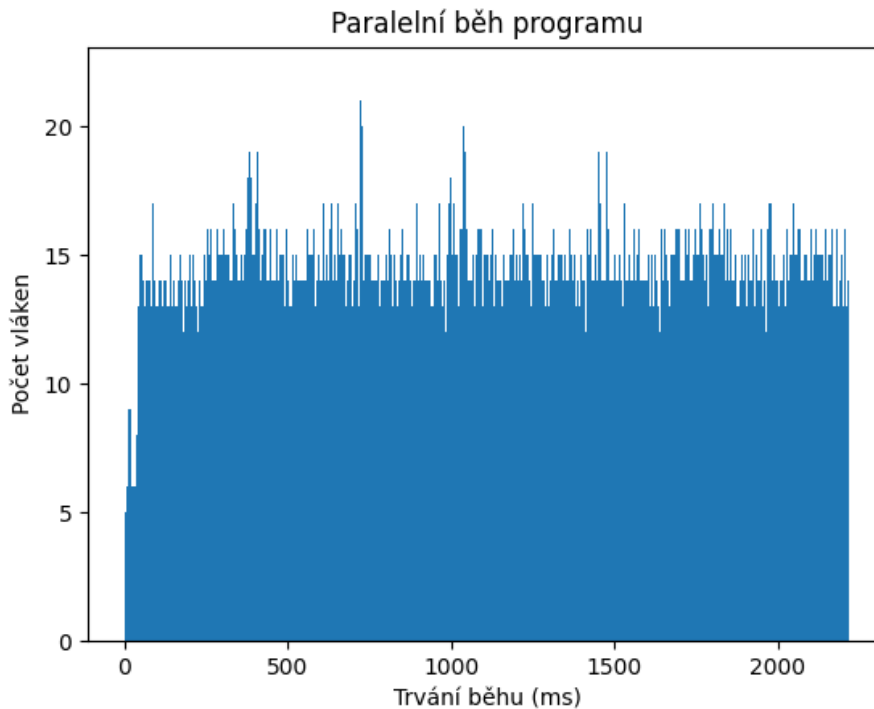
```

/**
 * Output Tensor for Single inference of Parallel processing.
 */
typedef struct OutParTensor
{
    std::vector<float> predictions; // Probabilities of output classes.
    float milliseconds = -1; // Elapsed milliseconds of inference.
    float offset_milliseconds = -1; // Elapsed milliseconds from the start of inference.
} OutParTensor;

/**
 * Output Tensor for All inferences of Parallel processing.
 */
typedef struct OutParTensors
{
    std::vector<OutParTensor> out_tensors; // All Output Tensors.
    float milliseconds = -1; // Elapsed milliseconds of all inferences.
} OutParTensors;
  
```

Zdrojový kód 4.4: Struktury pro měření času paralelním zpracováním

Z obrázku 4.4 lze vidět struktury, které agregují informace pro výsledek inference a měření času. Čas je měřen v milisekundách a poskytuje informace o běhu všech inferencí pro paralelní zpracování, ale také čas běhu každé inference zvlášť a času startu inference. Z těchto dat lze následně vytvořit graf hustoty, který ukazuje počet paralelních zpracování za jednotku času, viz obrázek 2.



Obrázek 2: Graf vytvořený pomocí Python skriptu zobrazující počet vláken za jednotku času

4.2.3 Měření času

Čas je zaznamenáván v milisekundách jako typ `float`, což umožňuje přesnější měření (mikrosekundy). Pro každé zařízení je využívána jiná strategie měření. Pro GPU se využívá zabudovaný systém pro měření událostí poskytovaný CUDA API v C++. Pro CPU je poté čas měřen pomocí standardní C++ hlavičky `std::chrono`.

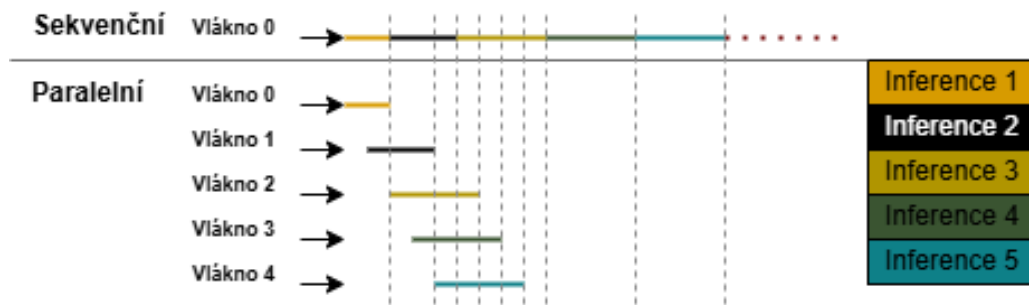
4.2.4 Frugally Deep

Knihovna Frugally Deep je pouze hlavičková knihovna, která poskytuje nízkou latenci inference, jednoduchost implementace, minimální závislosti a přeno-

sitelnost kódu.

Návrh

Program poskytuje metody jak pro jednoduchou sekvenční inferenci, tak pro paralelní zpracování více obrázků. Na obrázku 3 lze vidět jakým způsobem je paralelní zpracování prováděno:



Obrázek 3: Konkurence vláken na CPU

V samotném programu je paralelismus implementován pomocí standardních možností jazyka C++, a to konkrétně pomocí `std::future`. Postupně se tedy během běhu programu spouští jednotlivé vlákna a na konci jsou všechny spojeny do hlavního vlákna a běh programu končí.

Program po dokončení inference vygeneruje .csv soubor, který obsahuje data z paralelního běhu. Konkrétně obsahuje délku běhu inference, čas po kterém byla inference (vlákno) spuštěna a předpovězenou třídu inference. Z těchto dat byl pomocí vlastního Python skriptu vytvořen graf, který ukazuje počet spuštěných vláken za jednotku času 2.

Využití SYCL

V původním zadání bylo zmíněno, že by se Frugally Deep mohlo pomocí SYCL využívat na heterogenních prostředích, tz. CPU, GPU a FGPA. Tato varianta avšak byla ve výsledku změněna na využití TensorRT a ONNX-Runtime pro běh na heterogenním prostředí z několika důvodů:

1. Pro využití Frugally Deep uvnitř SYCL by bylo potřeba významně změnit fungování paměti uvnitř knihovny, což by nebylo vhodné zejména kvůli udržitelnosti kódu (updaty knihovny).
2. Modely umělé inteligence jsou nejčastěji spouštěny na NVidia GPU, které jsou nejvhodnější pro inferenci [18].

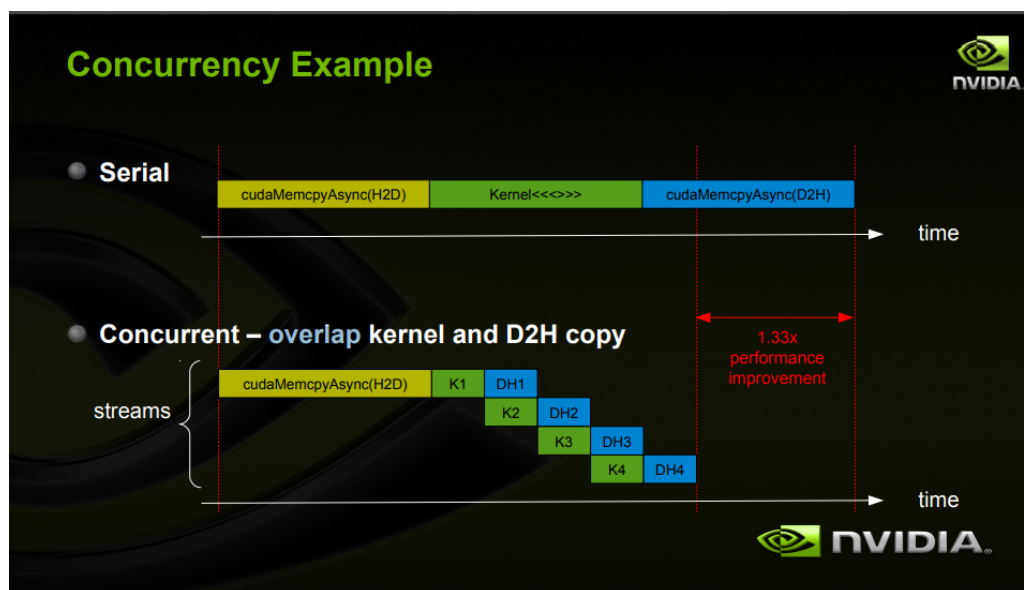
3. TensorRT a Onnx-Runtime jsou oproti obecným postupům vysoce optimalizovanými nástroji pro práci s neuronovými sítěmi při práci na GPU [20].

4.2.5 TensorRT

Jako jedna z knihoven pro inferenci na GPU byl vybrán TensorRT. TensorRT je knihovna poskytovaná firmou NVidia a vysoce optimalizovaná pro práci s neuronovými sítěmi a výpočty na GPU.

Návrh

Inferenční GPU část obsahuje také možnost sekvenční, nebo paralelní inference vstupů. TensorRT však řeší paralelismus jinak a to pomocí metody zvané overlapping [17]. Obrázky jsou tedy zpracovávány sekvenčně, ale zpracování každého obrázku je prováděno paralelně, viz obrázek 4.



Obrázek 4: Paralelismus TensorRT

Závěr

Akcelerace pomocí TensorRT byla implementována, jako jedno ze tří možných řešení spuštění. Nepodařilo se však implementaci plně optimalizovat (pravděpodobně z důvodu špatných knihoven).

4.2.6 Onnx Runtime

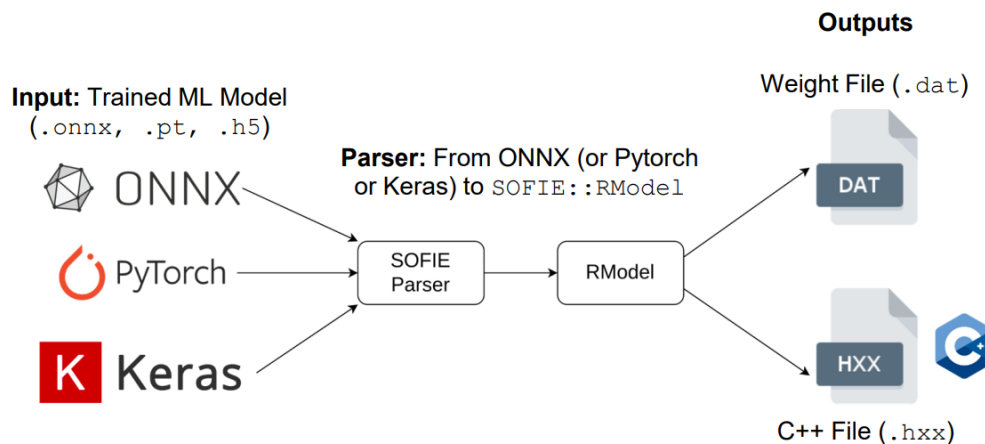
Jako poslední knihovna byla vybrána Onnx-Runtime. Byl vybrána především z důvodu, že poskytuje možnost akcelerace na různých platformách, které se dají snadno definovat jako takzvaný ExecutionProvider [10]. Inference lze tedy akcelarovat na CPU, dále pomocí Intel technologie OneDNN na Intel CPU [11]. Dále lze kód spustit, s drobnými změnami, na GPU a to pomocí CUDA [9] a TensorRT [13]. V neposlední řadě lze inference zprovoznit i pro platformu Android a jiné.

Návrh

Inferenční část Onnx-Runtime poskytuje prozatím pouze možnost sekvenčního inference. Avšak Onnx-Runtime by měl poskytovat možnost paralelizace celého procesu o velikosti nastaveného poolu. To tedy znamená, že v jeden časový úsek program automaticky spustí maximálně pevně nastavený počet vláken.

4.3 TMVA Sofie

Sofie byla vybrána nejen kvůli možnosti implementace jak pro CPU tak GPU, ale také protože měla nabízet rozšíření, které obsahovalo přesun inference do SYCL frameworku. Sofie má na vstupu natrénovaný model a výstupem kód in-



Obrázek 5: Graf znázorňující průběh Sofie [6]

ferenční funkce v jazyce C++. To má několik výhod jako je snadná integrace do jiných projektů v jazyce C++, minimální závislost (pouze na BLAS) a další [6].

4.3.1 Instalace

Sofie běží pod systémem ROOT, instalace je možná udělat jak ručně (pomocí předkompilovaných binárních souborů), tak pomocí správce balíčků. ROOT lze nainstalovat přímo ze správce balíčků operačního systému pro Linuxové distribuce jako NixOS/Nix/Nixpkgs, Gentoo, Arch Linux, CentOS a Fedora [25]. Pro Debian a Ubuntu ale zatím verze dostupná není. V dokumentaci je napsáno, že se na řešení již pracuje, mezitím je ROOT k dispozici v Ubuntu prostřednictvím nástroje Conda nebo pomocí předkompilovaných binárních souborů.

Instalace pomocí binárních souborů

V případě této instalace je třeba ručně doinstalovat potřebné závislosti.

```
sudo apt install binutils cmake dpkg-dev g++ gcc libssl-dev git libx11-dev \
libxext-dev libxft-dev libxpm-dev python3 libtbb-dev libvdt-dev libgif-dev
```

Zdrojový kód 4.5: Závislosti pro Ubuntu a další distribuce založené na Debianu

```

#install root
# need to get the right version for your system
# it is possible to do via package manager (but not for ubuntu)
wget https://root.cern/download/
root_v6.32.08.Linux-almalinux9.4-x86_64-gcc11.4.tar.gz
tar -xzf root_v6.32.08.Linux-almalinux9.4-x86_64-gcc11.4.tar.gz
source root/bin/thisroot.sh

# build root directories
git clone --branch latest-stable
--depth=1 https://github.com/root-project/root.git root_src
# prepare instalation folder
mkdir root_build
cd root_build
# make With selected options (turn on sofie)
cmake -Dtmva-sofie=On -Dtmva-cpu=On -DCMAKE_INSTALL_PREFIX=../root ../root_src
cmake --build . --target install
make -j8

```

Zdrojový kód 4.6: Instalace Sofie pro CPU pomocí předkompilovaných binárních souborů

Každá možnost sestavení je logická proměnná, kterou lze zapnout nebo vypnout. Uživatel může z příkazového řádku cmake nastavit libovolnou proměnnou, která řídí proces sestavování. Předáním příkazu `cmake -D <var>=<hodnota>` se vytvoří záznam v mezipaměti CMake [23]. S instalací pomocí binárních souborů vzniklo několik komplikací, které nejsou v dokumentaci ROOT popsány. Po dlouhém hledání bylo zjištěno, že ROOT zatím nepodporuje Tensorflow ve vyšší verzi než je 2.15. Na Tensorflow je ale Sofie přímo závislá, takže bez správné verze Tensorflow inference neběží. Tensorflow ve verzi 2.15 a vyšší není kompatibilní s Pythonem verzí vyšší než 3.11. Ubuntu 24.04, na kterém byla Sofie spouštěna již nepodporuje instalaci Pythonu 3.11 bez virtuálního prostředí. Pokud je ale Python nainstalován pomocí virtuálního prostředí, některé předkompilované binární soubory, které byly závislé na Pythonu přestanou fungovat a Sofie se nespustí.

Instalace pomocí Condy

Vzhledem ke komplikovanosti instalace pomocí binárních souborů byla zvolena instalace pomocí virtuálního prostředí Conda. Nejprve je potřeba nainstalovat Condu, v řešení je využita MiniConda.

```

mkdir -p ~/miniconda3
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh

```

```
-O ~/miniconda3/miniconda.sh
bash ~/miniconda3/miniconda.sh -b -u -p ~/miniconda3
rm ~/miniconda3/miniconda.sh
source ~/miniconda3/bin/activate
```

Zdrojový kód 4.7: Instalace MiniCondy

Pro vytvoření nového prostředí ROOT v Condě je třeba vytvořit prostředí a připravit správné verze Pythonu a Tensorflow, s kterými ROOT dokáže pracovat (Tím se odstraní problém závislostí, který nastal u předkompilovaných binárních souborů).

```
conda init --all
conda create -n my_root_env root -c conda-forge
conda activate my_root_env
conda config --env --add channels conda-forge
conda install python=3.11
pip install tensorflow==2.15
```

Zdrojový kód 4.8: Příprava ROOT prostředí v Condě

Takto připravené prostředí již spustí přípravu modelu v ROOT a inferenci na CPU bez problému. Sofie na to má již předpřipravené testovací kódy, které byly pro testování využity [24]. Pro přípravu spuštění na GPU je třeba připravit závislosti na architekturu Cuda. Vzhledem k nutnosti konkrétní verze Tensorflow (2.15) bylo třeba najít konkrétní podporované verze Cudy, které jsou třeba do virtuálního prostředí nainstalovat.

GPU

Version	Python version	Compiler	Build tools	cuDNN	CUDA
tensorflow-2.18.0	3.9-3.12	Clang 17.0.6	Bazel 6.5.0	9.3	12.5
tensorflow-2.17.0	3.9-3.12	Clang 17.0.6	Bazel 6.5.0	8.9	12.3
tensorflow-2.16.1	3.9-3.12	Clang 17.0.6	Bazel 6.5.0	8.9	12.3
tensorflow-2.15.0	3.9-3.11	Clang 16.0.0	Bazel 6.1.0	8.9	12.2

Obrázek 6: Verze Cudy [31]

```
conda install -c conda-forge tensorflow-gpu==2.15
conda install -c nvidia cuda-toolkit=12.2
conda install -c conda-forge "cudnn=8.9"
conda install nvidia/label/cuda-12.2.2::cuda
```

Zdrojový kód 4.9: Příprava závislostí pro spuštění inference na grafice

```
(my_root_env) ~/miniconda3/envs/my_root_env$ conda list tensorflow
#
# Name                               Version                               Build Channel
tensorflow                           2.15.0                               cuda120py311h5cbd639_3 conda-forge
tensorflow-base                       2.15.0                               cuda120py311h43b5e44_3 conda-forge
tensorflow-estimator                  2.15.0                               cuda120py311hf663016_3 conda-forge
tensorflow-gpu                        2.15.0                               cuda120py311hb76ca00_3 conda-forge

(my_root_env) conda list python
# packages in environment at
# Name                               Version                               Build Channel

python                               3.11.11                               h9e4cc4f_1_cpython conda-forge

(my_root_env) conda list cudnn
/home/kubala/miniconda3/envs/my_root_env:
#
# Name                               Version                               Build Channel
cudnn                                8.9.7.29                               h092f7fd_3 conda-forge
(my_root_env) conda list cuda
#
# Name                               Version                               Build Channel
cuda                                  12.2.2                                0 nvidia/label/cuda-12.2.2
```

Zdrojový kód 4.10: Kontrola verzí pro spuštění na GPU

4.3.2 Spuštění

Spuštění příkladové inference je pak pomocí několika příkazů.

```
root tutorials/tmva/TMVA_Higgs_Classification.C
python tutorials/tmva/TMVA_SOFIE_Inference.py
```

Zdrojový kód 4.11: Spuštění inference v Sofii

Testovací inference vypíše využití CPU a GPU, váhy, relu a další podrobnosti k běhu. Při úspěšné nastavení Sofie pro GPU se při výpisu zobrazí informace o využitém GPU.

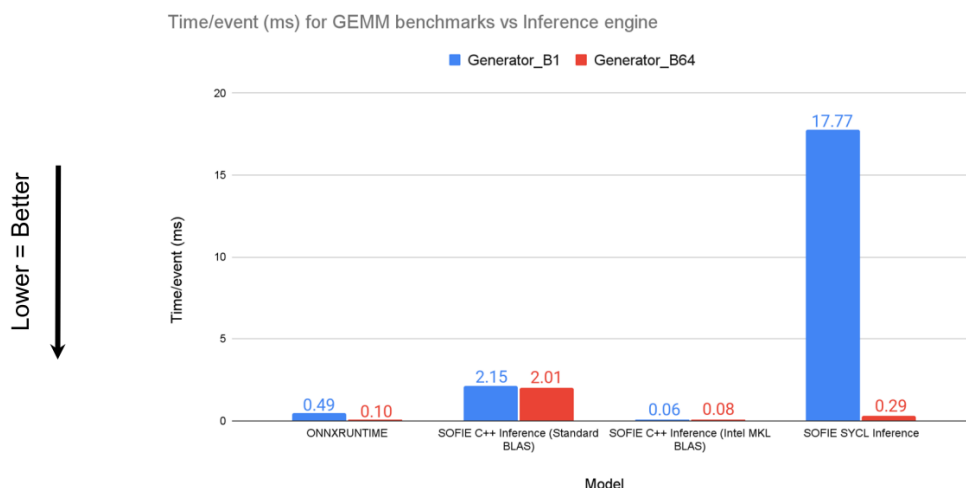
```
Created device /job:localhost/replica:0/task:0/device:GPU:0
with 4084 MB memory: -> device: 0, name: NVIDIA GeForce GTX 1660 SUPER,
pci bus id: 0000:06:00.0, compute capability: 7.5
```

Zdrojový kód 4.12: Informace o GPU z testovací inference Sofie

Výsledky inference Sofie (vygenerované soubory a výstupy) jsou součástí zdrojových souborů.

4.3.3 Sofie-SYCL

Hlavním důvodem proč zprovoznit Sofii byla Sofie-SYCL, při analýze bylo toto rozšíření vhodným adeptem pro řešení celého projektu. Sofie-SYCL rozšiřuje stávající funkce Sofie o generování inferenčního kódu SYCL, který lze spustit na grafických procesorech Intel pomocí knihoven Intel oneAPI MKL a na grafických procesorech Intel/NVidia/AMD pomocí knihoven portBLAS [27].



Obrázek 7: Výsledky z ROOT ukazující rychlost Sofie [26]

Problém se Sofií-SYCL je ten, že ještě není součástí TMVA řešení a je vytvořen pouze její prototyp [21].

Nejprve bylo třeba nastavit SYCL prostředí pomocí Intel® oneAPI DPC++ aplikace [29] [30]. Instalace by musela proběhnout pomocí předkompilovaných binárních souborů 4.3.1 se specifikací pro kompilátory SYCLu, jelikož Sofie-SYCL jiné řešení sestavení zatím nemá.

```
sudo cmake ../ -G "Unix Makefiles" -DSYCL_ACADEMY_USE_DPCPP=ON
-DSYCL_ACADEMY_ENABLE_SOLUTIONS=ON
-DCMAKE_C_COMPILER="/opt/intel/oneapi/compiler/2025.0/bin/icx"
-DCMAKE_CXX_COMPILER="/opt/intel/oneapi/compiler/2025.0/bin/icpx"
-DIntelSYCL_DIR="/opt/intel/oneapi/compiler/2025.0/lib/cmake/IntelSYCL"
-DTBB_DIR="/opt/intel/oneapi/tbb/latest/lib/cmake/tbb"
```

Zdrojový kód 4.13: Sestavení Sofie-SYCL

Problém nastal u linkování TBB závislostí. Knihovnu TBB vyžaduje i Sofie bez rozšíření. SYCL nedokáže zpracovat předkompilovanou knihovnu od ROOTu a Sofie nedokáže zatím pracovat s knihovnou TBB nainstalovanou lokálně pro Intel (typicky pod /opt/intel/oneapi/tbb/latest/lib/cmake/tbb).

Zkoumání TMVA Sofie pro inferenci na GPU bylo primárně motivováno jejím potenciálním SYCL backendem. Nicméně fakt, že je tento backend zatím pouze prototypem, který není součástí hlavní distribuce TMVA, představovala značné překážky. Zprovoznění SYCL rozšíření vyžaduje vyřešit spoustu komplikací se závislostmi a nastavením celého sestavení. Až bude Sofie-SYCL plně integrována do TMVA, mohla by nabídnout výkonné a snadno přenositelné řešení pro inferenci akceleroanou pomocí GPU.

5 Příručka

Tato kapitola obsahuje podrobný návod, jak zprovoznit jednotlivé knihovny pro správné spuštění programu.

5.1 Kompilace pro Frugally Deep

Ke zprovoznění knihovny Frugally Deep není zapotřebí žádné dodatečné instalace. Všechny potřebné knihovny, jako například frugally-deep, eigen, FunctionalPlus a json jsou již zahrnuty v řešení ve složce libs.

Program je nadále sestaven pomocí nástroje CMake s argumentem `-DACCELERATOR=1`.

5.2 Kompilace pro Onnx-Runtime s CUDA

Pro kompilaci knihovny Onnx-Runtime s CUDA Execution Providerem je zapotřebí nainstalovat CUDA [14], CuDNN [15], zdrojové soubory Onnx-Runtime [12].

Po stažení všech knihoven lze z adresáře knihovny Onnx-Runtime spustit příkaz:

```
./build.bat
--config RelWithDebInfo
--build_shared_lib
--parallel --use_cuda
--cuda_version 12.6
--cuda_home "<path_to_cuda_installation_home>"
--cudnn_home "<path_to_cudnn_installation_home>"
--skip_tests
```

Zdrojový kód 5.1: Sestavení knihovny Onnx-Runtime s CUDA

Nástroj CMake takto sestaví knihovnu Onnx-Runtime pro CPU Execution Provider a CUDA Execution Provider.

Pro sestavení programu je dále nutné zahrnout argument `-DACCELERATOR=3`, který značí, že kód bude kompilován pro Onnx-Runtime a dále `-DONNX_RUNTIME_ROOT=<path_to_onnx_runtime_installation>`, který je kořenovým adresářem právě sestavené knihovny.

5.3 Kompilace pro TensorRT

Poslední možností kompilace programu je kompilace se závislostmi na TensorRT. Nejdříve je třeba stáhnout soubor TensorRT [16] a extrahovat ho ze ZIP do požadovaného adresáře.

Pro sestavení programu je dále nutné zahrnout argument `-DACCELERATOR=2`, který značí, že kód bude kompilován pro TensorRT a dále `-DTENSOR_RT_ROOT=<path_to_tensor_rt_installation>`, která signalizuje kořenový adresář nainstalované knihovny.

6 Výsledky

Pro porovnání byl několikrát spuštěn program nad testovací sadou, která obsahuje 17484 obrázků pro inferenci. Program byl spuštěn celkem desetkrát pro každou inferenční knihovnu (Frugally Deep, TensorRT a ONNX Runtime) a zároveň podle možností sekvenčně i paralelně, viz tabulky 6.1, 6.2 a 6.3.

Tabulka 6.1: Naměřený čas pro spuštění Frugally Deep (ms)

	Sekvenční běh	Paralelní běh
	10400.3	2476.57
	10271.2	2522.6
	10580.6	2702.09
	10339.1	2506.93
	10387.9	2485.72
	11135.6	2490.07
	10494.6	2538.15
	10365.8	2722.65
	10352.7	2491.77
	11194.5	2498.07
Průměr	10552.2	2543.46

Tabulka 6.2: Naměřený čas pro spuštění Onnx Runtime (ms)

	Sekvenční běh	Paralelní běh
	7860.81	-
	8602.8	-
	7859.27	-
	7782.41	-
	7816.63	-
	7925.95	-
	7954.94	-
	7964.52	-
	7954.49	-
	7726.52	-
Průměr	7944.83	-

Tabulka 6.3: Naměřený čas pro spuštění TensorRT (ms)

	Sekvenční běh	Paralelní běh
	29697.7	52069.187500
	30186.5	51614.292969
	30166.2	52000.621094
	29576.7	52122.566406
	42708.2	63606.238281
	30243.3	56353.554688
	30750.6	53318.230469
	44018.0	64541.542969
	37168.1	61797.187500
	33792.5	54534.007812
Průměr	33830.8	56195.742188

Z tabulek 6.1, 6.2 a 6.3 lze vidět, že nejefektivnějším postupem pro inferenci je paralelní spuštění na CPU pomocí knihovny Frugally Deep. Inference celé sady obsahující 17484 obrázků byla provedena za přibližně 2.5 vteřiny.

Ze spuštění inference pomocí sekvenčního běhu bylo neefektivnější spuštění pomocí Onnx-Runtime, které bylo dokončeno za necelých 8 vteřin. Díky tomuto měření a teoretickým podkladům [18] lze tedy předpokládat zrychlení inference pomocí akcelerace na GPU oproti klasickému CPU.

Dalším předpokladem pro GPU pro budoucí využití je zvýšení efektivity pro hlubší a složitější neuronové sítě, u kterých je zapotřebí více výpočtů. V aktuálním modelu může být rychlost inference na GPU lehce snížena cenou přenosu dat na GPU vůči méně složitým výpočtům.

7 Závěr

V této práci je implementován inferenční systém využívající různé frameworky. Byly srovnány výkonnostní charakteristiky jednotlivých frameworků na vybraných modelech. Získané výsledky ukazují, že optimalizované frameworky pro NVidii jsou výkonnější. Mezi hlavní omezení patří nutnost přenosu dat na GPU při jednoduchých modelech.

Dalšími kroky jsou zapojení paralelního zpracování inference na frameworku ONNX Runtime a zhodnocení rychlostí při složitějších neuronových modelech (modely diplomové práce).

Literatura

- [1] AMD. *uProf* [online]. AMD. Dostupné z: <https://www.amd.com/en/developer/uprof.html>.
- [2] BAIR. *Caffe* [online]. Caffe. Dostupné z: <https://caffe.berkeleyvision.org/>.
- [3] DOBIASD. *Frugally Deep Source* [online]. Frugally Deep. Dostupné z: <https://github.com/Dobiasd/frugally-deep>.
- [4] FRUGALLYDEEP. *FrugallyDeep* [online]. FrugallyDeep. Dostupné z: <https://github.com/Dobiasd/frugally-deep>.
- [5] INTEL. *vtune* [online]. Intel. Dostupné z: <https://www.intel.com/content/www/us/en/developer/tools/oneapi/vtune-profiler.html#gs.iwmkkn>.
- [6] LORENZO MONETA, S. S. I. P. *Accelerating Machine Learning Inference on GPUs with SYCL using SOFIE* [online]. ROOT. Dostupné z: https://indico.cern.ch/event/1330797/contributions/5796633/attachments/2818114/4920403/TMVA_SOFIE_ACAT2024.pdf.
- [7] MAGICSE. *Tensorflow-onnx source* [online]. magicse. Dostupné z: <https://github.com/onnx/tensorflow-onnx>.
- [8] MICROSOFT. *Onnx Runtime AMD ROCm Execution Providers* [online]. microsoft. Dostupné z: <https://onnxruntime.ai/docs/execution-providers/ROCM-ExecutionProvider.html>.
- [9] MICROSOFT. *Onnx Runtime CUDA Execution Providers* [online]. microsoft. Dostupné z: <https://onnxruntime.ai/docs/execution-providers/CUDA-ExecutionProvider.html>.
- [10] MICROSOFT. *Onnx Runtime Execution Providers* [online]. microsoft. Dostupné z: <https://onnxruntime.ai/docs/execution-providers/>.
- [11] MICROSOFT. *Onnx Runtime Intel OneDNN Execution Providers* [online]. microsoft. Dostupné z: <https://onnxruntime.ai/docs/execution-providers/oneDNN-ExecutionProvider.html>.
- [12] MICROSOFT. *Onnx Runtime Sources* [online]. microsoft. Dostupné z: <https://github.com/microsoft/onnxruntime>.

- [13] MICROSOFT. *Onnx Runtime TensorRT Execution Providers* [online]. microsoft. Dostupné z: <https://onnxruntime.ai/docs/execution-providers/TensorRT-ExecutionProvider.html>.
- [14] NVIDIA. *CUDA Download* [online]. NVidia. Dostupné z: <https://developer.nvidia.com/cuda-downloads>.
- [15] NVIDIA. *CUDNN Download* [online]. NVidia. Dostupné z: <https://developer.nvidia.com/cudnn-downloads>.
- [16] NVIDIA. *TensorRT Download* [online]. NVidia. Dostupné z: <https://developer.nvidia.com/tensorrt/download/10x>.
- [17] NVIDIA. *TensorRT Parallelism* [online]. NVidia. Dostupné z: <https://developer.download.nvidia.com/CUDA/training/StreamsAndConcurrencyWebinar.pdf>.
- [18] DOMINANCE. *nvidia-dominance* [online]. nvidia-dominance. Dostupné z: <https://dl.acm.org/doi/fullHtml/10.1145/3631537>.
- [19] ONNXRUNTIME. *onnxRuntime* [online]. onnxRuntime. Dostupné z: <https://onnxruntime.ai/docs/>.
- [20] RESTACK. *sycllearn* [online]. Restack. Dostupné z: <https://www.restack.io/p/ai-optimization-answer-tensorrt-optimization-cat-ai>.
- [21] ROOT. *SofiePull* [online]. ROOT. Dostupné z: <https://github.com/root-project/root/pull/13550>.
- [22] ROOT. *clanekSofie* [online]. ROOT. Dostupné z: <https://zenodo.org/records/8385777>.
- [23] ROOT. *sofieBuild* [online]. ROOT. Dostupné z: https://root.cern/install/build_from_source/.
- [24] ROOT. *sofieInference* [online]. ROOT. Dostupné z: https://root.cern.ch/doc/master/TMVA__SOFIE__Inference_8py.html.
- [25] ROOT. *sofieIsntall* [online]. ROOT. Dostupné z: <https://root.cern/install/#install-via-a-package-manager>.
- [26] ROOT. *sofiePrezentace2* [online]. ROOT. Dostupné z: <https://indico.cern.ch/event/1311339/contributions/5534985/attachments/2698718/4683872/Inference%20of%20ML%20Models%20on%20Intel%20GPUs%20with%20SYCL%20and%20Intel%20OneAPI%20using%20SOFIE.pdf>.
- [27] ROOT. *sofieSycl* [online]. ROOT. Dostupné z: <https://github.com/joannapng/root/tree/sofie-sycl/tmva/sofie>.

- [28] SOFIE. *sofie* [online]. sofie. Dostupné z:
<https://github.com/root-project/root/tree/master/tmva/sofie>.
- [29] SYCL. *sycl* [online]. SYCL. Dostupné z:
<https://sycl.tech/getting-started>.
- [30] SYCL. *sycllearn* [online]. SYCL. Dostupné z:
<https://www.intel.com/content/www/us/en/developer/articles/event-contest/reinders-sycl-links-from-talk.html>.
- [31] TENSORFLOW. *tensorflowVersions* [online]. Tensorflow. Dostupné z:
<https://www.tensorflow.org/install/source>.
- [32] TENSORRT. *tensorRT* [online]. tensorRT. Dostupné z:
<https://docs.nvidia.com/deeplearning/tensorrt/>.
- [33] CNN. *tiny-cnn* [online]. tiny-cnn. Dostupné z:
<https://github.com/gcp/tiny-cnn>.
- [34] WAITINGSOFIESYCL. *waitingSofieSycl* [online]. waitingSofieSycl. Dostupné z:
<https://github.com/root-project/root/pull/13550>.